

Universidad Distrital FJDC – Facultad de Ingeniería – Ingeniería de Sistemas – taller No. 1. Mecanismo de Seguridad informática. Taller No. 1. Prof. Octavio J. Salcedo Parra. Septiembre 14 /2026. Entrega: septiembre 24/2026. octavionetworking@gmail.com

(Web Application and API Protection)
con Inteligencia Artificial / Machine Learning, RASP y DevSecOps

Curso sugerido	Mecanismos de Seguridad Informática / Seguridad de Aplicaciones y DevSecOps
Nivel	Pregrado avanzado / Posgrado (Especialización o Maestría en Ciberseguridad)
Modalidad	Laboratorio (implementación – Desarrollo), individual o en equipos máximo 4 estudiantes.
Duración estimada	12 a 16 horas (2–3 sesiones de laboratorio)
Prerrequisitos	Redes TCP/IP, HTTP/HTTPS, fundamentos de OWASP Top 10, línea de comandos Linux, nociones de Python y CI/CD
Entorno	Contenedores Docker / Docker Compose sobre Linux (Ubuntu 22.04+) o WSL2

Contenido

1. Presentación y Justificación	2
2. Objetivos	3
2.1 Objetivo general	3
2.2 Objetivos específicos	3
2.3 Competencias a desarrollar.....	3
3. Marco Conceptual	4
3.1 De WAF a WAAP: evolución del perímetro de aplicaciones	4
3.2 Componentes funcionales de un WAAP	4
3.3 Inteligencia Artificial y Machine Learning en WAAP	5
3.3.1 Características (features) típicas extraídas de una petición HTTP	5
3.3.2 Modelos sugeridos	5
3.3.3 Riesgos y limitaciones a discutir en el laboratorio	5
3.4 RASP (Runtime Application Self-Protection)	5
3.4.1 Ventajas frente al WAF/WAAP perimetral	5
3.4.2 Limitaciones	6
3.5 DevSecOps aplicado a la seguridad de aplicaciones	6
3.5.1 Controles DevSecOps abordados en el taller	6

3.6 Marcos normativos y de referencia	6
4. Arquitectura de Referencia del Laboratorio	7
5. Requisitos e Infraestructura del Taller	8
5.1 Requisitos de software	8
5.2 Requisitos de hardware sugeridos	8
5.3 Estructura sugerida del repositorio de trabajo	8
6. Desarrollo del Taller	9
Fase 0 — Preparación del entorno	9
Fase 1 — Despliegue de la aplicación objetivo vulnerable	10
Fase 2 — Implementación de la capa base WAF/WAAP (reglas)	10
Fase 3 — Módulo de detección basado en IA/ML	11
3.1. Recolección y etiquetado del tráfico	11
3.2. Extracción de características	11
3.3. Entrenamiento del modelo	11
3.4. Inferencia e integración con el orquestador de decisión	12
Fase 4 — Integración RASP (Runtime Application Self-Protection)	13
4.1. Diseño del agente RASP	13
4.2. Casos de prueba	13
Fase 5 — Pipeline DevSecOps	14
Ejercicio de verificación del pipeline	14
Fase 6 — Pruebas de efectividad y técnicas de evasión	15
6.1. Herramientas de ataque controlado	15
6.2. Técnicas de evasión sugeridas para el análisis	15
6.3. Matriz de resultados	15
Fase 7 — Observabilidad y métricas	16
7. Entregables del Taller	16
9 Referencias	17

1. Presentación y Justificación

Las aplicaciones web modernas y sus interfaces de programación (API) constituyen hoy la superficie de ataque más expuesta de cualquier organización. Los cortafuegos de aplicaciones web (WAF) tradicionales, basados exclusivamente en firmas y expresiones regulares, resultan insuficientes frente a amenazas que combinan

automatización a gran escala (bots), abuso de lógica de negocio, ataques a APIs REST/GraphQL y técnicas de evasión de reglas estáticas.

Gartner acuñó el término WAAP (Web Application and API Protection) para describir la convergencia de cuatro capacidades en una sola plataforma: protección WAF de nueva generación, mitigación de bots, seguridad de API y protección DDoS de capa 7, todo ello reforzado por motores de Inteligencia Artificial y Machine Learning (IA/ML) para la detección de comportamientos anómalos. A esta convergencia se suma, en arquitecturas de seguridad en profundidad, la protección en tiempo de ejecución (RASP, Runtime Application Self-Protection), que actúa desde dentro de la propia aplicación, y la integración con flujos DevSecOps, que llevan la seguridad al ciclo de vida de desarrollo de software (shift-left security).

Este taller propone que el estudiante diseñe, implemente y evalúe, en un entorno de laboratorio controlado, un prototipo funcional de WAAP que integre estos tres pilares: un motor de detección aumentado con IA/ML, un componente RASP embebido en la aplicación objetivo, y un pipeline DevSecOps que gestione el ciclo de vida seguro de las reglas y del propio código de la aplicación.

Nota metodológica

El taller está diseñado para un entorno exclusivamente académico y aislado (sandbox / red interna sin salida a Internet productiva). Todas las herramientas y técnicas de explotación se emplean únicamente contra la aplicación vulnerable desplegada por el propio estudiante, nunca contra sistemas de terceros.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar, en un entorno de laboratorio, un prototipo de plataforma WAAP que integre un módulo de detección basado en Inteligencia Artificial/Machine Learning, un componente de protección en tiempo de ejecución (RASP) y un pipeline DevSecOps, evaluando su efectividad frente a ataques web comunes documentados en el OWASP Top 10 y el OWASP API Security Top 10.

2.2 Objetivos específicos

1. Caracterizar las diferencias funcionales y arquitectónicas entre un WAF tradicional y una plataforma WAAP de nueva generación.
2. Desplegar una capa base de filtrado mediante un motor de reglas de código abierto (Coraza o ModSecurity) con el OWASP Core Rule Set (CRS).
3. Construir y entrenar un modelo de Machine Learning para la detección de anomalías en el tráfico HTTP a partir de características extraídas de las peticiones.
4. Instrumentar la aplicación objetivo con un agente RASP capaz de interceptar y bloquear operaciones peligrosas en tiempo de ejecución (por ejemplo, construcción insegura de consultas SQL).
5. Diseñar un pipeline CI/CD con controles DevSecOps (SAST, SCA, análisis de contenedores e infraestructura como código) que valide el código antes de su despliegue.
6. Evaluar de forma comparativa la tasa de detección, los falsos positivos y la resiliencia frente a técnicas de evasión de cada capa de protección (reglas, IA/ML y RASP) de forma individual y combinada.

2.3 Competencias a desarrollar

- Análisis crítico de arquitecturas de seguridad perimetral y de aplicación.

- Aplicación práctica de algoritmos de detección de anomalías a datos de seguridad.
- Diseño de controles de seguridad automatizados dentro de un flujo de integración continua.
- Pensamiento adversarial: evaluación de un control de seguridad desde la perspectiva del atacante.

3. Marco Conceptual

3.1 De WAF a WAAP: evolución del perímetro de aplicaciones

El modelo de protección perimetral clásico asume que el tráfico malicioso puede identificarse mediante patrones conocidos. Sin embargo, el crecimiento de las arquitecturas de microservicios, el consumo masivo de APIs y la sofisticación de los bots automatizados exigen un modelo de defensa en capas, dinámico y consciente del contexto de negocio. La siguiente tabla resume las diferencias principales.

Característica	WAF tradicional	WAAP (nueva generación)
Base de detección	Firmas y expresiones regulares estáticas	Firmas + IA/ML + análisis de comportamiento
Alcance	Tráfico HTTP/HTTPS genérico	Web, API REST/GraphQL/gRPC, microservicios
Gestión de bots	Limitada o inexistente	Bot management dedicado (huella digital, biometría de comportamiento)
Adaptabilidad	Reglas manuales, alta fricción de mantenimiento	Modelos que se reentrenan con tráfico real
Protección DDoS L7	Generalmente no incluida	Integrada nativamente
Despliegue	Appliance / proxy perimetral	Nube, edge (CDN), sidecar, o embebido (RASP)
Integración con desarrollo	Nula (silo de seguridad de red)	Integrada en pipelines DevSecOps (shift-left)

3.2 Componentes funcionales de un WAAP

Un WAAP moderno se compone típicamente de los siguientes módulos, todos ellos abordados en este taller:

- **Motor de reglas (Core Rule Engine):** conjunto de firmas y reglas declarativas (p. ej. OWASP CRS) que detectan patrones conocidos de inyección SQL, XSS, path traversal, etc.
- **Módulo de IA/ML:** modelos estadísticos o de aprendizaje automático que perfilan el comportamiento «normal» del tráfico y señalan desviaciones (anomalías), permitiendo detectar variantes de ataque no cubiertas por firmas.
- **Seguridad de API:** validación de esquemas (OpenAPI/JSON Schema), control de versiones, límites de tasa (rate limiting) y detección de API «fantasma» o no documentada (shadow APIs).
- **Gestión de bots:** diferenciación entre tráfico humano y automatizado mediante huellas de dispositivo, análisis de cabeceras TLS/JA3 y patrones de interacción.
- **RASP (Runtime Application Self-Protection):** agente embebido dentro del proceso de la aplicación que observa y controla operaciones sensibles (consultas a bases de datos, llamadas al sistema, deserialización) con el contexto completo de la ejecución.

- **Integración DevSecOps: automatización del ciclo de vida de las reglas y del propio código mediante pipelines CI/CD con puertas de calidad de seguridad.**

3.3 Inteligencia Artificial y Machine Learning en WAAP

La componente de IA/ML de un WAAP puede abordarse mediante aprendizaje supervisado (clasificación de peticiones como benignas o maliciosas a partir de tráfico etiquetado) o no supervisado (detección de anomalías sin necesidad de etiquetas, asumiendo que el tráfico de ataque es estadísticamente distinto del tráfico normal). En este taller se emplea un enfoque no supervisado, más realista para un entorno donde no siempre se dispone de tráfico de ataque previamente etiquetado.

3.3.1 Características (features) típicas extraídas de una petición HTTP

- Longitud de la URL y del cuerpo de la petición.
- Entropía de Shannon del payload (los payloads ofuscados o codificados tienden a mayor entropía).
- Número y longitud de parámetros de consulta.
- Presencia de metacaracteres sospechosos (' , --, <script>, ../, |, ;).
- Frecuencia de peticiones por IP/sesión en una ventana temporal (para detectar automatización).
- Método HTTP, tipo de contenido y cabeceras atípicas (User-Agent, Accept, Referer).

3.3.2 Modelos sugeridos

- Isolation Forest: aísla observaciones atípicas mediante particiones aleatorias; eficiente y con buen desempeño en alta dimensionalidad. Utilizado como modelo principal del taller.
- One-Class SVM: aprende la frontera de la clase «normal» en un espacio de características; sensible a la escala de los datos.
- Autoencoders (redes neuronales): reconstruyen la entrada y usan el error de reconstrucción como score de anomalía; adecuados para secuencias de tráfico más complejas.

3.3.3 Riesgos y limitaciones a discutir en el laboratorio

- Falsos positivos: tráfico legítimo pero inusual (campañas de marketing, picos estacionales) puede marcarse como anómalo.
- Ataques adversariales: un atacante puede fragmentar o mimetizar su tráfico para acercarlo estadísticamente al tráfico normal (evasión por mimicry).
- Envenenamiento del modelo (data poisoning): si el reentrenamiento usa tráfico ya contaminado con ataques de bajo volumen, el modelo puede «normalizar» el ataque.
- Explicabilidad: la dificultad de justificar por qué el modelo bloqueó una petición concreta, relevante para auditoría y cumplimiento.

3.4 RASP (Runtime Application Self-Protection)

A diferencia del WAF/WAAP perimetral, que analiza el tráfico de red antes de llegar a la aplicación, el RASP se instrumenta dentro del propio proceso de la aplicación (mediante agentes, hooks de bytecode, interceptores de framework o extensiones del intérprete). Esto le permite observar el contexto de ejecución real: la consulta SQL final tal como se construye internamente, la ruta de archivo que realmente se va a abrir, o el objeto que se va a deserializar, incluso si el payload llegó ofuscado o fragmentado y evadió el filtrado perimetral.

3.4.1 Ventajas frente al WAF/WAAP perimetral

- Visibilidad del contexto de ejecución real (post-decodificación, post-normalización).

- Independencia del cifrado de transporte y de técnicas de evasión basadas en codificación de la petición.
- Capacidad de bloquear la operación peligrosa concreta (p. ej. la llamada a la base de datos) en lugar de solo la petición HTTP.

3.4.2 Limitaciones

- Impacto en el rendimiento de la aplicación (overhead de instrumentación).
- Acoplamiento al lenguaje/framework de la aplicación (un agente RASP para Java no protege una aplicación en Python sin un agente equivalente).
- Requiere despliegue y mantenimiento por cada servicio, a diferencia de un WAAP centralizado.

3.5 DevSecOps aplicado a la seguridad de aplicaciones

El enfoque DevSecOps traslada los controles de seguridad al ciclo de vida de desarrollo de software (shift-left), automatizando su ejecución dentro del pipeline de integración y despliegue continuo (CI/CD). En el contexto de un WAAP, esto se traduce en dos frentes complementarios:

- Seguridad del código y la infraestructura que soportan el WAAP y la aplicación (SAST, SCA, análisis de contenedores, IaC scanning).
- Gestión como código (policy as code) de las propias reglas del WAAP y de los umbrales del modelo de IA/ML, versionadas, revisadas por pares y desplegadas automáticamente.

3.5.1 Controles DevSecOps abordados en el taller

Control	Propósito	Herramienta sugerida (código abierto)
SAST	Análisis estático del código fuente en busca de patrones inseguros	Semgrep / Bandit (Python)
SCA	Detección de dependencias con vulnerabilidades conocidas (CVE)	pip-audit / OWASP Dependency-Check
Container scanning	Detección de vulnerabilidades en imágenes de contenedor	Trivy
IaC scanning	Validación de configuraciones inseguras en definiciones de infraestructura	Checkov
DAST	Pruebas dinámicas contra la aplicación en ejecución	OWASP ZAP (modo baseline/CI)
Policy as code	Umbrales y reglas del WAAP versionadas en el repositorio	Archivos YAML/Rego versionados en Git

3.6 Marcos normativos y de referencia

- OWASP Top 10 (2021) y OWASP API Security Top 10: catálogo de riesgos que orienta las reglas y los casos de prueba del laboratorio.
- MITRE ATT&CK: en particular la técnica T1190 (Exploit Public-Facing Application), para documentar el mapeo de los ataques simulados.
- NIST SP 800-53 (controles SC-7 «Boundary Protection» y SI-4 «System Monitoring»): referencia para justificar la arquitectura de defensa en profundidad.
- PCI DSS, requisito 6.4.2: exige protección automatizada de aplicaciones web orientadas al público, marco habitual para justificar la adopción de WAF/WAAP en entornos con datos de tarjetas de pago.

- CIS Controls v8, control 16 (Application Software Security).

4. Arquitectura de Referencia del Laboratorio

La arquitectura propuesta organiza el laboratorio en tres planos que interactúan entre sí: el plano de datos (tráfico HTTP/HTTPS hacia la aplicación objetivo, filtrado por la capa WAAP y por el agente RASP embebido), el plano de control (el pipeline DevSecOps que valida, construye y despliega tanto el código de la aplicación como las reglas y el modelo de IA/ML), y el plano de observabilidad (logging centralizado, métricas y, opcionalmente, un SIEM para correlación de eventos).

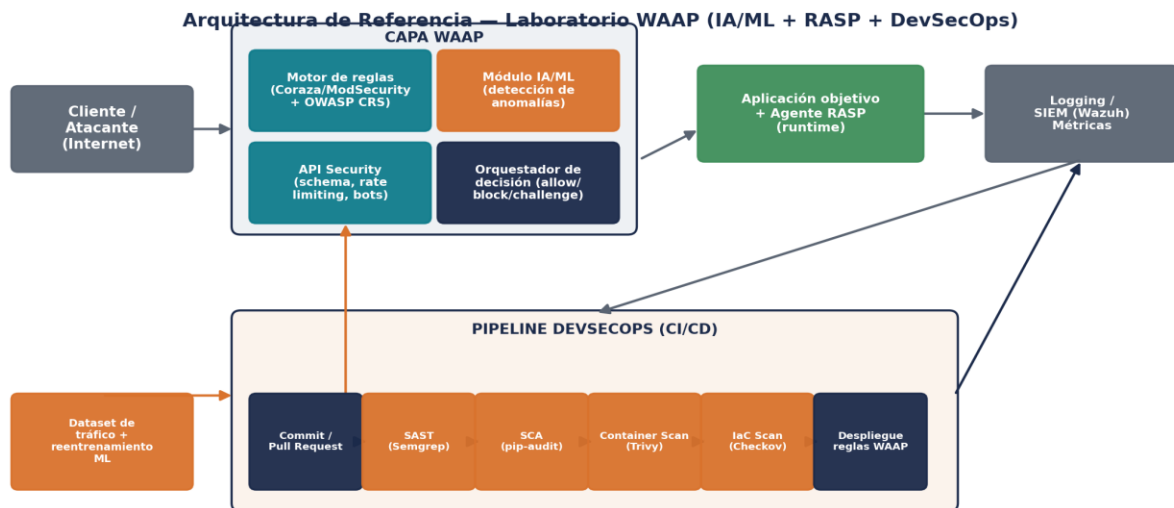


Figura 1. Arquitectura de referencia del laboratorio WAAP.

El flujo de una petición HTTP recorre los siguientes puntos de decisión, en orden:

1. El cliente (legítimo o atacante) envía una petición hacia la aplicación objetivo.
2. El motor de reglas (Coraza/ModSecurity + OWASP CRS) evalúa la petición contra firmas conocidas.
3. En paralelo, el módulo de IA/ML calcula un score de anomalía a partir de las características de la petición.
4. Un orquestador de decisión combina el resultado de ambas capas (reglas + score de anomalía) y decide: permitir, bloquear o desafiar (challenge, por ejemplo un CAPTCHA simulado).
5. Si la petición se permite, llega a la aplicación, donde el agente RASP intercepta las operaciones sensibles (por ejemplo, la construcción de una consulta SQL) y puede bloquearlas en tiempo de ejecución aunque hayan superado las capas anteriores.
6. Todos los eventos (permitidos, bloqueados, alertados) se envían al subsistema de logging/SIEM, que además alimenta el reentrenamiento periódico del modelo de IA/ML.
7. El pipeline DevSecOps valida cualquier cambio en el código de la aplicación, en las reglas del WAAP o en el modelo, antes de que llegue a este entorno.

5. Requisitos e Infraestructura del Taller

5.1 Requisitos de software

Componente	Herramienta propuesta	Función en el laboratorio
Sistema operativo	Ubuntu 22.04 LTS (nativo o WSL2)	Entorno base del laboratorio
Contenerización	Docker + Docker Compose	Orquestación de los servicios del laboratorio
Aplicación objetivo	OWASP Juice Shop o DVWA	Aplicación web deliberadamente vulnerable
Motor WAF	Coraza (Go) u OWASP ModSecurity + CRS	Capa base de filtrado por firmas
Lenguaje del módulo IA/ML	Python 3.10+, scikit-learn, pandas, numpy	Entrenamiento e inferencia del modelo de anomalías
RASP	Middleware/agente propio en Python, o referencia a OpenRASP (Java/PHP)	Protección en tiempo de ejecución
CI/CD	GitHub Actions o GitLab CI (runner local)	Pipeline DevSecOps
SAST	Semgrep o Bandit	Análisis estático de código
SCA	pip-audit u OWASP Dependency-Check	Análisis de dependencias
Escaneo de contenedores	Trivy	Vulnerabilidades en imágenes Docker
Escaneo de IaC	Checkov	Configuraciones inseguras en Dockerfile/Compose/Terraform
Pruebas de ataque	OWASP ZAP, sqlmap, curl	Generación de tráfico de ataque controlado
Logging / SIEM (opcional)	Wazuh o ELK Stack	Correlación y visualización de eventos

5.2 Requisitos de hardware sugeridos

- CPU: 4 núcleos como mínimo (recomendado 8 para ejecutar el pipeline CI/CD y el entrenamiento del modelo en paralelo).
- RAM: 8 GB mínimo, 16 GB recomendado.
- Almacenamiento: 20 GB libres para imágenes de contenedor, datasets y artefactos del pipeline.
- Red: red interna aislada (bridge de Docker); no se requiere salida a Internet productiva salvo para la descarga inicial de imágenes y paquetes.

5.3 Estructura sugerida del repositorio de trabajo

```
taller-waap/
├─ app/                # Aplicación objetivo (Juice Shop/DVWA) + agente RASP
```

```

├── rasp_agent.py
├── ...
├── waap/
│   ├── coraza/          # Configuración del motor de reglas + OWASP CRS
│   └── ml/
│       ├── train_model.py
│       ├── extract_features.py
│       └── model.pkl
├── pipeline/
│   └── .github/workflows/devsecops.yml
├── logs/
│   └── access.log
├── docker-compose.yml
└── README.md

```

6. Desarrollo del Taller

El taller se organiza en siete fases progresivas. Cada fase incluye el objetivo, los pasos guiados y un producto verificable. Se recomienda documentar evidencias (capturas de pantalla, logs, código) de cada fase en el informe final.

Fase 0 — Preparación del entorno

Objetivo: levantar la infraestructura base del laboratorio sobre contenedores.

```

# 1. Crear la red y el proyecto
mkdir taller-waap && cd taller-waap

# 2. docker-compose.yml (fragmento base)
version: '3.8'
services:
  juice-shop:
    image: bkimminich/juice-shop
    ports: ['3000:3000']
    networks: [lab-net]

  waap-proxy:
    image: owasp/modsecurity-crs:nginx
    ports: ['8080:8080']
    depends_on: [juice-shop]
    networks: [lab-net]

networks:
  lab-net:
    driver: bridge

# 3. Levantar el entorno
docker compose up -d
docker compose ps

```

Producto verificable: acceso a la aplicación objetivo a través del proxy WAAP (<http://localhost:8080>) y evidencia de los contenedores en ejecución.

Fase 1 — Despliegue de la aplicación objetivo vulnerable

Objetivo: contar con una aplicación con vulnerabilidades documentadas (inyección SQL, XSS, control de acceso roto) que sirva como blanco controlado para todas las fases posteriores.

1. Desplegar OWASP Juice Shop (o DVWA) según el docker-compose.yml de la Fase 0.
2. Explorar el mapa de vulnerabilidades incluido en la propia aplicación (Juice Shop expone un «Score Board» con retos categorizados por tipo de vulnerabilidad OWASP).
3. Registrar como línea base al menos 5 vulnerabilidades que se explotarán en las fases 3 a 6 (por ejemplo: inyección SQL en el login, XSS reflejado en la búsqueda, IDOR en el acceso a pedidos).

Entregable de la Fase 1

Tabla con las vulnerabilidades seleccionadas, su categoría OWASP Top 10 / API Security Top 10 y el endpoint específico donde se explotan.

Fase 2 — Implementación de la capa base WAF/WAAP (reglas)

Objetivo: desplegar el motor de reglas con el OWASP Core Rule Set como primera línea de defensa, y verificar su capacidad de bloqueo ante ataques básicos.

1. Configurar el nivel de paranoia (Paranoia Level, PL1 a PL4) del CRS; se recomienda iniciar en PL1 para reducir falsos positivos y aumentar progresivamente.
2. Enviar una petición con un payload de inyección SQL clásico y verificar en los logs que el motor de reglas genera una alerta y/o un bloqueo (HTTP 403).
3. Repetir la prueba con una técnica de evasión simple (por ejemplo, comentarios SQL en línea o codificación de caracteres) y documentar si el motor de reglas la detecta o no.

```
# Prueba de detección básica
curl -i "http://localhost:8080/rest/products/search?q=' OR '1'='1"

# Prueba de evasión (codificación de caracteres)
curl -i "http://localhost:8080/rest/products/search?q=%27%20OR%20%271%27%3D%271"

# Revisar el log de auditoría del motor de reglas
tail -f /var/log/modsec_audit.log
```

Entregable de la Fase 2

Evidencia de al menos un bloqueo exitoso y un caso de evasión exitosa (que el motor de reglas no detectó), como base comparativa para la Fase 3.

Fase 3 — Módulo de detección basado en IA/ML

Objetivo: construir un modelo de detección de anomalías entrenado con tráfico normal, capaz de señalar peticiones estadísticamente atípicas, incluyendo variantes de ataque que evadieron el motor de reglas en la Fase 2.

3.1. Recolección y etiquetado del tráfico

Generar tráfico normal navegando manualmente por la aplicación objetivo (búsquedas, inicio de sesión, navegación de catálogo) durante al menos 15–20 minutos, registrando cada petición en un archivo de log estructurado (URL, cuerpo, cabeceras, marca de tiempo, IP).

3.2. Extracción de características

```
# extract_features.py – extracción de características de un log de acceso
import re, math, pandas as pd

def shannon_entropy(s):
    if not s:
        return 0.0
    probs = [s.count(c) / len(s) for c in set(s)]
    return -sum(pr * math.log2(pr) for pr in probs)

SUSPICIOUS = re.compile(r"(--|;|<script|\\.\\.|\\bUNION\\b|\\bOR\\b\\s+1=1)", re.I)

def extract(request_row):
    url = request_row['url']
    body = request_row.get('body', '')
    return {
        'url_length': len(url),
        'body_length': len(body),
        'entropy': shannon_entropy(url + body),
        'n_params': url.count('&') + 1 if '?' in url else 0,
        'has_suspicious_chars': bool(SUSPICIOUS.search(url + body)),
        'req_per_minute': request_row.get('req_per_minute', 0),
    }
```

3.3. Entrenamiento del modelo

```
# train_model.py – entrenamiento del modelo de detección de anomalías
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import joblib

df = pd.read_csv('logs/features_normal_traffic.csv') # solo tráfico normal
X = df[['url_length', 'body_length', 'entropy', 'n_params',
        'has_suspicious_chars', 'req_per_minute']].astype(float)

scaler = StandardScaler().fit(X)
X_scaled = scaler.transform(X)

model = IsolationForest(
    n_estimators=200, contamination=0.02, random_state=42
).fit(X_scaled)
```

```
joblib.dump({'model': model, 'scaler': scaler}, 'waap/ml/model.pkl')
print('Modelo entrenado y guardado en waap/ml/model.pkl')
```

3.4. Inferencia e integración con el orquestador de decisión

```
# score_request.py – inferencia sobre una petición entrante
import joblib, pandas as pd
bundle = joblib.load('waap/ml/model.pkl')
model, scaler = bundle['model'], bundle['scaler']

def score(features: dict) -> float:
    X = pd.DataFrame([features])
    X_scaled = scaler.transform(X)
    # decision_function: valores negativos => más anómalo
    return float(model.decision_function(X_scaled)[0])

ANOMALY_THRESHOLD = -0.05

def is_anomalous(features: dict) -> bool:
    return score(features) < ANOMALY_THRESHOLD
```

1. Ejecutar el modelo contra el mismo payload de evasión que superó el motor de reglas en la Fase 2 y registrar el score de anomalía obtenido.
2. Ajustar el umbral (ANOMALY_THRESHOLD) para equilibrar la tasa de falsos positivos frente a la tasa de detección, documentando al menos dos configuraciones distintas.
3. Simular un escenario de bot/automatización (ráfaga de peticiones en segundos) y verificar si la característica req_per_minute permite su detección.

Entregable de la Fase 3

Modelo entrenado (model.pkl), matriz de resultados con al menos 10 peticiones (5 normales, 5 de ataque/evasión) y su score de anomalía, y una breve discusión sobre falsos positivos observados.

Fase 4 — Integración RASP (Runtime Application Self-Protection)

Objetivo: instrumentar la aplicación objetivo con un agente que intercepte operaciones sensibles en tiempo de ejecución, demostrando protección incluso cuando un payload logra atravesar el motor de reglas y el módulo de IA/ML.

4.1. Diseño del agente RASP

Para fines académicos, el agente se implementa como un decorador/middleware que envuelve las funciones de la aplicación encargadas de construir consultas SQL, resolver rutas de archivo o deserializar objetos, e inspecciona el resultado final antes de que se ejecute.

```
# rasp_agent.py – middleware RASP simplificado (Flask/WSGI) para la app objetivo
import re, functools, logging
from flask import request, abort

SQLI_PATTERN = re.compile(r"(\bUNION\b|\bOR\b\s+1=1|--|;\s*DROP\b)", re.I)
logger = logging.getLogger('rasp')

def rasp_guard_query(query_builder_func):
    """Decorador que envuelve la construcción real de la consulta SQL
    dentro de la aplicación, con visibilidad del valor final ya
    concatenado -- después de cualquier decodificación previa."""
    @functools.wraps(query_builder_func)
    def wrapper(*args, **kwargs):
        final_query = query_builder_func(*args, **kwargs)
        if SQLI_PATTERN.search(final_query):
            logger.warning(
                'RASP: consulta SQL bloqueada en tiempo de ejecucion: %s',
                final_query,
            )
            abort(403, description='Operacion bloqueada por RASP')
        return final_query
    return wrapper

@rasp_guard_query
def build_login_query(username, password):
    # Ejemplo deliberadamente vulnerable a nivel de construccion de la
    # consulta; el RASP intercepta el resultado final antes de ejecutarlo.
    return f"SELECT * FROM users WHERE user='{username}' AND pass='{password}'"
```

4.2. Casos de prueba

1. Enviar directamente al backend (sin pasar por el WAAP) un payload de inyección SQL fragmentado en múltiples parámetros que individualmente no disparan el motor de reglas, pero que al concatenarse en la consulta final sí constituyen una inyección; verificar que el RASP la detecta.
2. Repetir el ataque de evasión de la Fase 2 (el que superó el motor de reglas) directamente contra la aplicación y verificar si el RASP lo bloquea en tiempo de ejecución.
3. Medir el impacto en la latencia de la aplicación con y sin el agente RASP activo (mínimo 20 peticiones por escenario).

Entregable de la Fase 4

Evidencia (logs) de al menos un bloqueo por RASP de un payload que había evadido las capas anteriores, y una tabla comparativa de latencia con/sin RASP.

Fase 5 — Pipeline DevSecOps

Objetivo: construir un pipeline CI/CD que valide automáticamente el código de la aplicación, sus dependencias, la imagen de contenedor y la configuración de infraestructura, bloqueando el despliegue si se detectan hallazgos críticos, y que sólo entonces despliegue de forma automática las reglas actualizadas del WAAP.

```
# .github/workflows/devsecops.yml
name: devsecops-pipeline
on: [push, pull_request]

jobs:
  security-gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: SAST - Semgrep
        run: |
          pip install semgrep
          semgrep --config=p/owasp-top-ten app/ --error

      - name: SCA - pip-audit
        run: |
          pip install pip-audit
          pip-audit -r app/requirements.txt

      - name: Build de la imagen de la aplicacion
        run: docker build -t taller-waap/app:${{ github.sha }} ./app

      - name: Container scan - Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'taller-waap/app:${{ github.sha }}'
          severity: 'CRITICAL,HIGH'
          exit-code: '1'

      - name: IaC scan - Checkov
        uses: bridgecrewio/checkov-action@master
        with:
          directory: .
          quiet: true

      - name: DAST - OWASP ZAP baseline
        uses: zaproxy/action-baseline@v0.12.0
        with:
          target: 'http://localhost:8080'

      - name: Despliegue de reglas WAAP (solo si todo pasa)
        if: success()
        run: ./scripts/deploy_waap_rules.sh
```

Ejercicio de verificación del pipeline

1. Ejecutar el pipeline con el código actual y verificar que todas las etapas pasan (línea base «verde»).

- Introducir deliberadamente una vulnerabilidad conocida (por ejemplo, una dependencia desactualizada con un CVE público, o una consulta SQL concatenada sin parametrizar) y verificar en qué etapa el pipeline la detiene.
- Revertir el cambio y confirmar que el pipeline vuelve a estado «verde» y ejecuta el despliegue de reglas.

Entregable de la Fase 5

Captura del pipeline fallando por la vulnerabilidad inyectada (indicando la etapa específica) y captura del pipeline en verde tras la corrección.

Fase 6 — Pruebas de efectividad y técnicas de evasión

Objetivo: evaluar de forma comparativa la capacidad de detección de cada capa (reglas, IA/ML, RASP) de forma individual y combinada, empleando herramientas estándar de la industria.

6.1. Herramientas de ataque controlado

- OWASP ZAP en modo de escaneo activo contra la instancia protegida por el WAAP completo.
- sqlmap contra los endpoints de inyección SQL identificados en la Fase 1, comparando resultados con y sin cada capa activa.
- Scripts propios de fuzzing ligero (por ejemplo, con listas de payloads de PayloadsAllTheThings) para generar variantes de XSS y SQLi.

6.2. Técnicas de evasión sugeridas para el análisis

- Codificación de caracteres (URL-encoding doble, Unicode).
- División de payloads entre múltiples parámetros o cabeceras.
- Comentarios en línea dentro de sentencias SQL para romper firmas basadas en cadenas exactas.
- Alternancia de mayúsculas/minúsculas y espacios en blanco atípicos (tabulaciones, saltos de línea codificados).

6.3. Matriz de resultados

Se recomienda documentar los resultados en una matriz como la siguiente (a completar por el estudiante):

Payload / técnica	Bloqueado por reglas (Fase 2)	Detectado por IA/ML (Fase 3)	Bloqueado por RASP (Fase 4)
SQLi clásico ' OR '1'='1	Sí / No	Sí / No	Sí / No
SQLi con codificación URL	Sí / No	Sí / No	Sí / No
SQLi fragmentado en 2 parámetros	Sí / No	Sí / No	Sí / No
XSS reflejado básico	Sí / No	Sí / No	Sí / No
Ráfaga de peticiones (bot)	Sí / No	Sí / No	N/A
Entregable de la Fase 6 Matriz de resultados completa y un análisis escrito (mínimo media página) que identifique			

Payload / técnica	Bloqueado por reglas (Fase 2)	Detectado por IA/ML (Fase 3)	Bloqueado por RASP (Fase 4)
qué combinación de capas ofreció la mejor cobertura y por qué.			

Fase 7 — Observabilidad y métricas

Objetivo: cerrar el ciclo de defensa en profundidad con visibilidad centralizada de los eventos de seguridad y métricas que permitan justificar decisiones de ajuste (tuning) de las reglas y del modelo.

- Centralizar los logs del motor de reglas, del módulo de IA/ML y del agente RASP en un único flujo (archivo estructurado JSON o, de forma opcional, Wazuh/ELK).
- Calcular, a partir de la matriz de la Fase 6, la tasa de verdaderos positivos, falsos positivos y falsos negativos por capa.
- Definir al menos un indicador de tiempo medio de detección (MTTD) simulado, comparando el momento del ataque con el momento del registro de la alerta.

Entregable de la Fase 7

Panel o tabla resumen de métricas (tasa de detección, falsos positivos, MTTD estimado) y una recomendación final de ajuste de umbrales.

7. Entregables del Taller

Al finalizar el laboratorio, cada estudiante o equipo debe entregar:

1. Informe técnico que documente las siete fases, incluyendo evidencias (capturas, logs, fragmentos de código propios).
2. Repositorio de código (comprimido o enlace a control de versiones) con: configuración del motor de reglas, script de extracción de características, script de entrenamiento del modelo (model.pkl incluido), agente RASP y definición del pipeline CI/CD.
3. Matriz de resultados de la Fase 6 completamente diligenciada.
4. Panel/tabla de métricas de la Fase 7.
5. Respuestas al cuestionario de análisis final (sección 9).

8. Cuestionario de Análisis Final

Responder de forma argumentada, con base en la evidencia obtenida durante el laboratorio:

1. Explique, con un ejemplo propio del laboratorio, una situación en la que el motor de reglas (firmas) fue insuficiente pero el módulo de IA/ML sí detectó la anomalía. ¿Por qué ocurrió esta diferencia?
2. Describa un escenario en el que el módulo de IA/ML podría generar un falso positivo sobre tráfico legítimo de la aplicación utilizada. ¿Qué característica (feature) sería la principal responsable?
3. ¿Por qué el agente RASP pudo detectar un ataque que había evadido tanto el motor de reglas como el módulo de IA/ML? Relacione su respuesta con el concepto de «contexto de ejecución».
4. Si tuviera que proteger una arquitectura de microservicios con 15 servicios distintos, ¿qué combinación de WAAP centralizado y RASP recomendaría, considerando el costo operativo de mantener agentes RASP en cada servicio?

5. Analice qué etapa del pipeline DevSecOps habría detenido, en un entorno productivo real, la vulnerabilidad que usted inyectó deliberadamente en la Fase 5. ¿Qué hubiera ocurrido si esa etapa no hubiese existido?
6. Relacione al menos dos de las vulnerabilidades explotadas en el laboratorio con técnicas específicas del MITRE ATT&CK (o su equivalente en OWASP API Security Top 10 si se trató de una API).
7. Proponga una mejora concreta (no mencionada en el taller) para reducir los falsos positivos del módulo de IA/ML sin perder capacidad de detección.
8. Desde una perspectiva de gobierno de datos, ¿qué riesgo implica reentrenar el modelo de IA/ML directamente con el tráfico capturado en producción sin ninguna validación previa?

9 Referencias

Gartner. *Market Guide for Cloud Web Application and API Protection*.

OWASP Foundation. OWASP Top 10:2021. <https://owasp.org/Top10/>

OWASP Foundation. OWASP API Security Top 10. <https://owasp.org/API-Security/>

OWASP Foundation. OWASP Core Rule Set (CRS). <https://coreruleset.org/>

OWASP Foundation. OWASP Juice Shop. <https://owasp.org/www-project-juice-shop/>

MITRE. ATT&CK Framework — T1190: Exploit Public-Facing Application. <https://attack.mitre.org/>

National Institute of Standards and Technology (NIST). SP 800-53 Rev. 5 — Security and Privacy Controls for Information Systems and Organizations.

PCI Security Standards Council. PCI DSS v4.0, requisito 6.4.2.

Center for Internet Security (CIS). CIS Controls v8 — Control 16: Application Software Security.

Coraza WAF Project. Documentación oficial. <https://coraza.io/>

Trivy — Aqua Security. Documentación oficial. <https://aquasecurity.github.io/trivy/>

Checkov — Bridgecrew/Prisma Cloud. Documentación oficial. <https://www.checkov.io/>

Semgrep. Documentación oficial. <https://semgrep.dev/>

scikit-learn developers. Isolation Forest — User Guide. <https://scikit-learn.org/>

Nota Importante: El taller No. 1, a desarrollar (resolver), no se está pidiendo (solicitando) ingeniera(o), que vulnere, ataque o hackea ninguno de los escenarios domésticos o empresariales a los cuales Usted pueda tener acceso. El ejercicio es carácter académico. Los participantes deben actuar como hackers éticos, sin causar daño intencional a sistemas (redes de comunicaciones) y personas. No se permite el uso de técnicas que comprometan la integridad de sistemas (redes de comunicaciones y servidores TCP/IP) reales sin permiso o autorización expresa de los propietarios de los escenarios en cuestión.